Learning with AI — Topic 2

Raspberry Pi Camera and GPIO Integration for On-Device AI Deployment

Aaryans Nepal · CSC 494 — IoT (Spring 2026)

What I Wanted to Learn

Deploy a working AI pipeline on a **Raspberry Pi 5** — not just make it compile, but get:

- A live **camera feed** into a Python script
- A **physical alert** (buzzer on a GPIO pin) wired to a software state machine
- Remote access that works from anywhere

All on-device, no cloud.

Lesson 1 — The Camera Connector Trap

Pi 5 has a **smaller CSI connector** than older Pi models. The Pi Camera Module 2 I started with used the older 15-pin cable.

> "Just buy an adapter." — everyone, incorrectly

I bought multiple 15-to-22-pin adapter ribbons. None worked.

With **Prof. Kenneth Ross's** help in his office, we systematically ruled out cables, then the Pi itself. The **Module 2 camera was the failing component**, not the cable.

Switched to a **Pi Camera Rev 1.3 (OV5647)** — plugs straight into the Pi 5. Live feed was working the same afternoon.

Takeaway: when three "fixes" fail on the same variable, *stop debugging the cable and change the component*.

Lesson 2 — Test Before Hardware Arrives

While waiting on a (never-working) adapter, I built a fallback: feed **pre-recorded phone video** through the same `cv2.VideoCapture` path as the live camera.

```
cam = cv2.VideoCapture(video_path or 0)   # file or camera
```

Result: half the pipeline — pose estimation, scoring, state machine, web stream — was **fully validated before the real camera worked**.

Design rule I keep now: if your vision pipeline doesn't care whether frames come from a file or a camera, you can ship most of it before hardware arrives.

Lesson 3 — gpiozero Makes the Buzzer Trivial

```
from gpiozero import Buzzer
buzzer = Buzzer(17)       # BCM GPIO 17, physical pin 11
buzzer.on()               # pull high
buzzer.off()
```

Total GPIO code in the whole project: about 10 lines.

What was *not* trivial: the state machine around it. A single bad frame can't fire the buzzer. You need **sustained** bad posture (10 s continuous) before alerting, and **sustained** good posture (3 s continuous) to silence.

The hardware is easy. The logic around *when* to pulse the hardware is where real work lives.

Lesson 4 — ZeroTier Is Not the Public Internet

Set up **ZeroTier** so I could SSH into the Pi from anywhere — great.

Tried to share `http://<ZeroTier-IP>:5000` in class sprint 1 demo. Got "access denied."

A ZeroTier IP is only reachable by devices on the same ZeroTier network. It is not public. For public access you need port forwarding, a tunnel (Cloudflare, ngrok), or a public IP.

Obvious in hindsight.

Lesson 5 — Sensor Tuning Matters

The Pi Camera Rev 1.3 (OV5647) behaves **differently** from the Module 2 (IMX219):

- Slower auto-white-balance convergence
- Warmer color rendition under indoor light
- Different libcamera tuning file (`ov5647.json`)

The code that "just works" on one Pi camera can look visibly wrong on another. **Know what sensor you're actually running on.**

Takeaways

- **Pi 5 CSI** connector is smaller than older Pi models — adapters exist, but so does switching cameras entirely.
- **Ship on files** while waiting on hardware — the pipeline doesn't know the difference.
- **gpiozero** is simple; state machines are the hard part.
- **ZeroTier ≠ public IP.** If other people need to see it, it needs a real route.
- **Camera sensors are not interchangeable** — even "both are Pi Cameras."

Thank You

Source: https://github.com/AaryansNepal/CSC-494_Learning-with-AI